

Enterprise AI Engineering Interview Handbook

Building an Enterprise HR AI Copilot with .NET, Azure & Semantic Kernel

This quick reference contains the most important interview topics and model answers based on the HR AI Copilot project. It is intended for Senior .NET AI Engineer interviews.

1. Explain your AI architecture.

Angular UI → ASP.NET Core API → AI Orchestrator → Semantic Kernel → Plugins → SQL Server / Azure AI Search / Blob Storage → Azure OpenAI. Business services fetch trusted data; the LLM performs reasoning and generates the final response.

2. Why Semantic Kernel?

It provides orchestration, plugin/function calling, chat history, prompt templates, streaming support and multi-LLM integration, reducing custom orchestration code.

3. Explain RAG.

Documents are uploaded to Blob Storage, chunked, converted to embeddings, indexed in Azure AI Search, retrieved through vector search, and injected into the prompt before calling the LLM.

4. What are embeddings?

Embeddings are numerical vectors representing semantic meaning. Azure AI Search compares the question embedding with document embeddings to retrieve the most relevant chunks.

5. Why Azure AI Search instead of SQL?

SQL performs keyword matching. Azure AI Search performs semantic/vector search, enabling natural-language questions to find relevant content even when exact words differ.

6. Plugin vs AI Agent.

A plugin performs a single business capability (for example GetLeaveBalance). An AI Agent coordinates multiple steps such as retrieving employee data, searching policies, reasoning, and producing a recommendation.

7. Why Multi-LLM?

To improve reliability, reduce vendor lock-in, optimize cost, and provide automatic fallback when one provider is unavailable.

8. How do you secure AI?

The LLM never accesses SQL directly. It calls authorized plugins. Document-level authorization filters Azure AI Search results, prompts are controlled, and every interaction is audited.

9. How do you optimize AI cost?

Track prompt/completion tokens, optimize chunk size and Top-K, cache embeddings, stream responses, and route simple requests to lower-cost models.

10. Explain production readiness.

Implement audit logs, Application Insights, retries, circuit breakers, background workers, streaming, rate limiting, usage quotas, prompt versioning, and monitoring.